

Practical Work no.1 – Documentation



Specification

We implement an abstract data type representing a directed graph with integer indexed vertices in the range $0 - (n-1)$

The implementation follows a layered architecture, separating:

- Domain layer : internal graph representation
- Repository layer : loads, saves in files and generates random graphs
- Service layer : exposes graph operations
- UI layer : console interface for interacting with the graph

Classes used

GraphData(int nrVertices)

- Constructs a graph with nrVertices vertices and no edges

Private:

`_nrVertices` = number of vertices

`_outEdges[v]` = list of vertices reachable from a vertex v

`_inEdges[v]` = list of vertices that have edges into v

`_costs[(u,v)]` = integer cost associated with edge $u - v$

Public:

`int nrVertices() const` = returns the number of vertices

`int nrEdges() const` = returns the number of edges

`inEdges()`
`outEdges()`
`costs()` } = accessors for adjacency lists and cost map

GraphData copy() const = creates a copy of the graph, duplicating all adjacency lists and cost mappings

GraphRepo(const string& filename)

- Loads and writes a graph in/from a text file. Also generates random graphs

Private:

filename = file from which the graph is loaded

graph = internal graph representation

void loadFile() = reads graph from the file specified in the constructor

Public:

GraphRepo(const string& filename) = loads a graph from the given file

void addDict(GraphData& g, int start, int end, int cost) = inserts an edge into the internal structure

- edge must not already exist

void removeDict(GraphData& g, int start, int end) = removes an edge from the internal structure

static void writeRandom(const string& filename, int vertices, int edges = -1) = creates a random graph and writes it to a file

- validates that $\text{edges} \leq \text{vertices}^2$

void saveModifications(const GraphData& g, const string& filename) = writes the current graph to a modifications file

GraphService(GraphRepo& repo, const string& modFile)

- Provides the public graph operations required by the ADT

Private:

repo = reference to the repository storing the graph

modFile = file where modifications are saved

Public:

`int getVerticesNumber() const` = returns the number of vertices

`vector<int> iterateVertices() const` = returns a sorted list of all vertices (iterator)

`bool checkEdge(int start, int end) const` = checks whether an edge exists or not

`int getInDegree(int v) const` = return inbound degree

`int getOutDegree(int v) const` = return outbound degree

`vector<int> iterateOutbound(int v) const` = return a sorted list of outbound neighbors

`vector<int> iterateInbound(int v) const` = return a sorted list of inbound neighbors

`int retrieveCost(int s, int e)` = returns cost of an existing edge

`void changeCost(int s, int e, int newCost)` = updates cost of existing edge

`void addVertex(int v)` = adds a new vertex

▪ `v` == current number of vertices

`void removeEdge(int s, int e)` = removes an edge

`GraphData copyGraph() const` = returns a copy of the graph

`const unordered_map<int, vector<int>>& get_in_edges() const`

`const unordered_map<int, vector<int>>& get_out_edges() const`

`const map<pair<int,int>, int>& get_costs() const`



used by UI to print internal representation

GraphUI

- Provides a console interface for interacting with the graph

Private:

`service` = pointer to the active service layer

Public:

`void start()` = displays initial menu

1. Load graph from file
2. Generate random graph

void loadGraph() = asks for input and modifications file, loads the graph through the repo and initializes the service

void generateGraph() = asks for number of vertices, optional number of edges, output and modifications file

- Validates edge count
- Generates a random graph

void menu() = displays the main command menu

1. number of vertices
2. iterate vertices
3. check edge
4. in degree
5. out degree
6. iterate inbound
7. iterate outbound
8. add vertex
9. remove vertex
10. add edge
11. remove edge
12. get cost
13. change cost
14. copy graph
15. show internal representation of graph
16. exit